

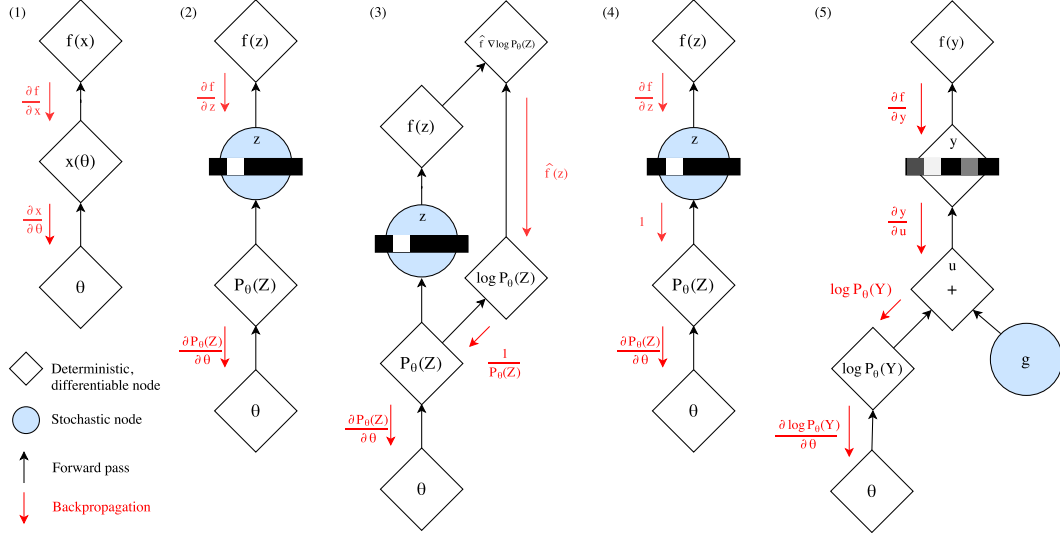


Figure 2: Gradient estimation in stochastic computation graphs. (1) $\nabla_{\theta} f(x)$ can be computed via backpropagation if $x(\theta)$ is deterministic and differentiable. (2) The presence of stochastic node z precludes backpropagation as the sampler function does not have a well-defined gradient. (3) The score function estimator and its variants (NVIL, DARN, MuProp, VIMCO) obtain an unbiased estimate of $\nabla_{\theta} f(x)$ by backpropagating along a surrogate loss $\hat{f} \log p_{\theta}(z)$, where $\hat{f} = f(x) - b$ and b is a baseline for variance reduction. (4) The Straight-Through estimator, developed primarily for Bernoulli variables, approximates $\nabla_{\theta} z \approx 1$. (5) Gumbel-Softmax is a path derivative estimator for a continuous distribution y that approximates z . Reparameterization allows gradients to flow from $f(y)$ to θ . y can be annealed to one-hot categorical variables over the course of training.

Gumbel-Softmax avoids this problem because each sample y is a differentiable proxy of the corresponding discrete sample z .

3.2 SCORE FUNCTION-BASED GRADIENT ESTIMATORS

The score function estimator (SF, also referred to as REINFORCE (Williams, 1992) and likelihood ratio estimator (Glynn, 1990)) uses the identity $\nabla_{\theta} p_{\theta}(z) = p_{\theta}(z) \nabla_{\theta} \log p_{\theta}(z)$ to derive the following unbiased estimator:

$$\nabla_{\theta} \mathbb{E}_z [f(z)] = \mathbb{E}_z [f(z) \nabla_{\theta} \log p_{\theta}(z)] \quad (5)$$

SF only requires that $p_{\theta}(z)$ is continuous in θ , and does not require backpropagating through f or the sample z . However, SF suffers from high variance and is consequently slow to converge. In particular, the variance of SF scales linearly with the number of dimensions of the sample vector (Rezende et al., 2014a), making it especially challenging to use for categorical distributions.

The variance of a score function estimator can be reduced by subtracting a control variate $b(z)$ from the learning signal f , and adding back its analytical expectation $\mu_b = \mathbb{E}_z [b(z) \nabla_{\theta} \log p_{\theta}(z)]$ to keep the estimator unbiased:

$$\nabla_{\theta} \mathbb{E}_z [f(z)] = \mathbb{E}_z [f(z) \nabla_{\theta} \log p_{\theta}(z) + (b(z) \nabla_{\theta} \log p_{\theta}(z) - b(z) \nabla_{\theta} \log p_{\theta}(z))] \quad (6)$$

$$= \mathbb{E}_z [(f(z) - b(z)) \nabla_{\theta} \log p_{\theta}(z)] + \mu_b \quad (7)$$

We briefly summarize recent stochastic gradient estimators that utilize control variates. We direct the reader to Gu et al. (2016) for further detail on these techniques.

- NVIL (Mnih & Gregor, 2014) uses two baselines: (1) a moving average $\bar{f}$ of f to center the learning signal, and (2) an input-dependent baseline computed by a 1-layer neural network

fitted to $f - \bar{f}$ (a control variate for the centered learning signal itself). Finally, variance normalization divides the learning signal by $\max(1, \sigma_f)$, where σ_f^2 is a moving average of $\text{Var}[f]$.

- DARN (Gregor et al., 2013) uses $b = f(\bar{z}) + f'(\bar{z})(z - \bar{z})$, where the baseline corresponds to the first-order Taylor approximation of $f(z)$ from $f(\bar{z})$. z is chosen to be $1/2$ for Bernoulli variables, which makes the estimator biased for non-quadratic f , since it ignores the correction term μ_b in the estimator expression.
- MuProp (Gu et al., 2016) also models the baseline as a first-order Taylor expansion: $b = f(\bar{z}) + f'(\bar{z})(z - \bar{z})$ and $\mu_b = f'(\bar{z})\nabla_{\theta}\mathbb{E}_z[z]$. To overcome backpropagation through discrete sampling, a mean-field approximation $f_{MF}(\mu_{\theta}(z))$ is used in place of $f(z)$ to compute the baseline and derive the relevant gradients.
- VIMCO (Mnih & Rezende, 2016) is a gradient estimator for multi-sample objectives that uses the mean of other samples $b = 1/m \sum_{j \neq i} f(z_j)$ to construct a baseline for each sample $z_i \in z_{1:m}$. We exclude VIMCO from our experiments because we are comparing estimators for single-sample objectives, although Gumbel-Softmax can be easily extended to multi-sample objectives.

3.3 SEMI-SUPERVISED GENERATIVE MODELS

Semi-supervised learning considers the problem of learning from both labeled data $(x, y) \sim \mathcal{D}_L$ and unlabeled data $x \sim \mathcal{D}_U$, where x are observations (i.e. images) and y are corresponding labels (e.g. semantic class). For semi-supervised classification, Kingma et al. (2014) propose a variational autoencoder (VAE) whose latent state is the joint distribution over a Gaussian “style” variable z and a categorical “semantic class” variable y (Figure 6, Appendix). The VAE objective trains a discriminative network $q_{\phi}(y|x)$, inference network $q_{\phi}(z|x, y)$, and generative network $p_{\theta}(x|y, z)$ end-to-end by maximizing a variational lower bound on the log-likelihood of the observation under the generative model. For labeled data, the class y is observed, so inference is only done on $z \sim q(z|x, y)$. The variational lower bound on labeled data is given by:

$$\log p_{\theta}(x, y) \geq -\mathcal{L}(x, y) = \mathbb{E}_{z \sim q_{\phi}(z|x, y)} [\log p_{\theta}(x|y, z)] - KL[q(z|x, y) || p_{\theta}(y)p(z)] \quad (8)$$

For unlabeled data, difficulties arise because the categorical distribution is not reparameterizable. Kingma et al. (2014) approach this by marginalizing out y over all classes, so that for unlabeled data, inference is still on $q_{\phi}(z|x, y)$ for each y . The lower bound on unlabeled data is:

$$\log p_{\theta}(x) \geq -\mathcal{U}(x) = \mathbb{E}_{z \sim q_{\phi}(y, z|x)} [\log p_{\theta}(x|y, z) + \log p_{\theta}(y) + \log p(z) - q_{\phi}(y, z|x)] \quad (9)$$

$$= \sum_y q_{\phi}(y|x) (-\mathcal{L}(x, y) + \mathcal{H}(q_{\phi}(y|x))) \quad (10)$$

The full maximization objective is:

$$\mathcal{J} = \mathbb{E}_{(x, y) \sim \mathcal{D}_L} [-\mathcal{L}(x, y)] + \mathbb{E}_{x \sim \mathcal{D}_U} [-\mathcal{U}(x)] + \alpha \cdot \mathbb{E}_{(x, y) \sim \mathcal{D}_L} [\log q_{\phi}(y|x)] \quad (11)$$

where α is the scalar trade-off between the generative and discriminative objectives.

One limitation of this approach is that marginalization over all k class values becomes prohibitively expensive for models with a large number of classes. If D, I, G are the computational cost of sampling from $q_{\phi}(y|x)$, $q_{\phi}(z|x, y)$, and $p_{\theta}(x|y, z)$ respectively, then training the unsupervised objective requires $\mathcal{O}(D + k(I + G))$ for each forward/backward step. In contrast, Gumbel-Softmax allows us to backpropagate through $y \sim q_{\phi}(y|x)$ for single sample gradient estimation, and achieves a cost of $\mathcal{O}(D + I + G)$ per training step. Experimental comparisons in training speed are shown in Figure 5.

4 EXPERIMENTAL RESULTS

In our first set of experiments, we compare Gumbel-Softmax and ST Gumbel-Softmax to other stochastic gradient estimators: Score-Function (SF), DARN, MuProp, Straight-Through (ST), and

Slope-Annealed ST. Each estimator is evaluated on two tasks: (1) structured output prediction and (2) variational training of generative models. We use the MNIST dataset with fixed binarization for training and evaluation, which is common practice for evaluating stochastic gradient estimators (Salakhutdinov & Murray, 2008; Larochelle & Murray, 2011).

Learning rates are chosen from $\{3e-5, 1e-5, 3e-4, 1e-4, 3e-3, 1e-3\}$; we select the best learning rate for each estimator using the MNIST validation set, and report performance on the test set. Samples drawn from the Gumbel-Softmax distribution are continuous during training, but are discretized to one-hot vectors during evaluation. We also found that variance normalization was necessary to obtain competitive performance for SF, DARN, and MuProp. We used sigmoid activation functions for binary (Bernoulli) neural networks and softmax activations for categorical variables. Models were trained using stochastic gradient descent with momentum 0.9.

4.1 STRUCTURED OUTPUT PREDICTION WITH STOCHASTIC BINARY NETWORKS

The objective of structured output prediction is to predict the lower half of a 28×28 MNIST digit given the top half of the image (14×28). This is a common benchmark for training stochastic binary networks (SBN) (Raiko et al., 2014; Gu et al., 2016; Mnih & Rezende, 2016). The minimization objective for this conditional generative model is an importance-sampled estimate of the likelihood objective, $\mathbb{E}_{h \sim p_\theta(h_i | x_{\text{upper}})} \left[\frac{1}{m} \sum_{i=1}^m \log p_\theta(x_{\text{lower}} | h_i) \right]$, where $m = 1$ is used for training and $m = 1000$ is used for evaluation.

We trained a SBN with two hidden layers of 200 units each. This corresponds to either 200 Bernoulli variables (denoted as 392-200-200-392) or 20 categorical variables (each with 10 classes) with binarized activations (denoted as 392-(20×10)-(20×10)-392).

As shown in Figure 3, ST Gumbel-Softmax is on par with the other estimators for Bernoulli variables and outperforms on categorical variables. Meanwhile, Gumbel-Softmax outperforms other estimators on both Bernoulli and Categorical variables. We found that it was not necessary to anneal the softmax temperature for this task, and used a fixed $\tau = 1$.

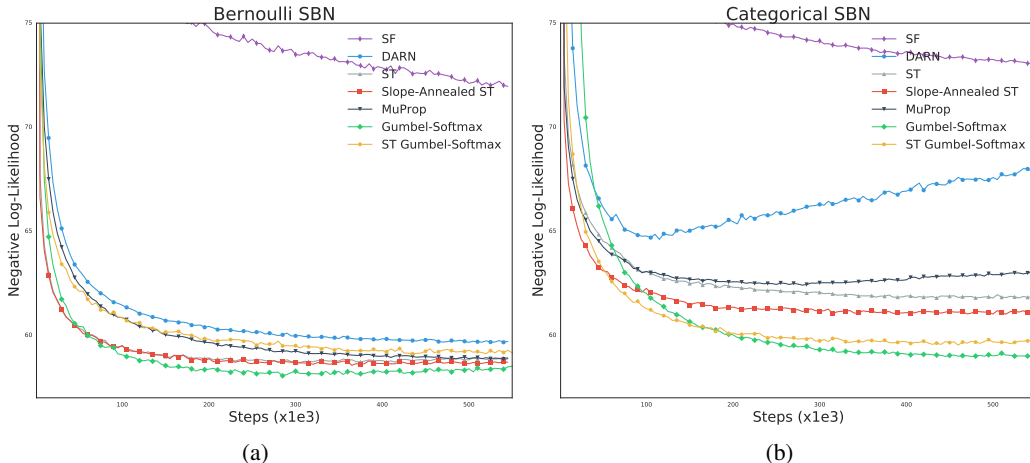


Figure 3: Test loss (negative log-likelihood) on the structured output prediction task with binarized MNIST using a stochastic binary network with (a) Bernoulli latent variables (392-200-200-392) and (b) categorical latent variables (392-(20×10)-(20×10)-392).

4.2 GENERATIVE MODELING WITH VARIATIONAL AUTOENCODERS

We train variational autoencoders (Kingma & Welling, 2013), where the objective is to learn a generative model of binary MNIST images. In our experiments, we modeled the latent variable as a single hidden layer with 200 Bernoulli variables or 20 categorical variables (20×10). We use a learned categorical prior rather than a Gumbel-Softmax prior in the training objective. Thus, the minimization objective during training is no longer a variational bound if the samples are not discrete. In practice,